Transformer Decoder Architecture

In the first part of this series, we explored the Transformer Encoder, which is essential for capturing complex patterns of input data. However, for tasks like machine translation, text generation, and other sequence-to-sequence applications, the Transformer Decoder plays a crucial role. The decoder generates meaningful output sequences by leveraging both the encoder's representations and the previously generated outputs. It achieves this through a combination of masked self-attention, cross-attention with the encoder, and feed-forward networks. In this blog, we'll dive into the architecture of the Transformer Decoder and how it enables powerful sequence generation across various applications.

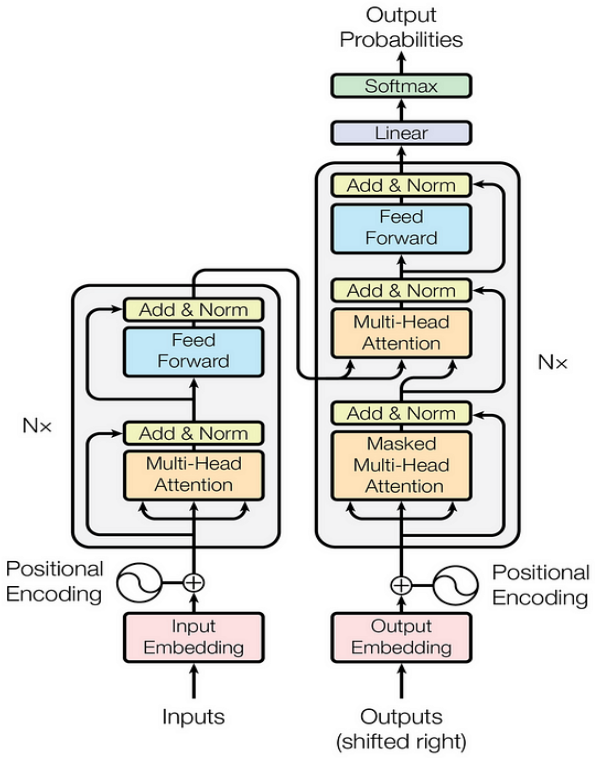
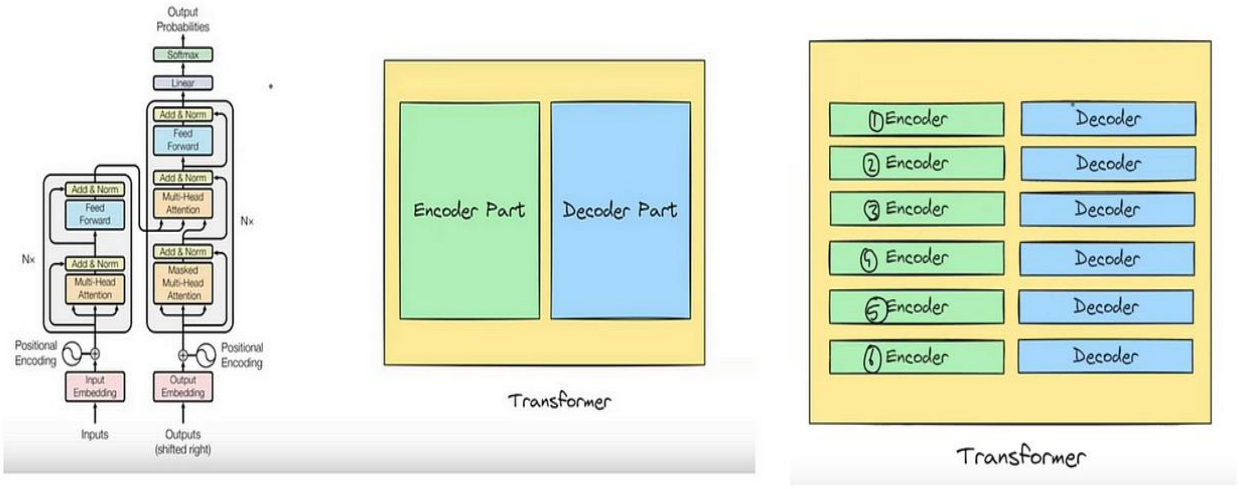


Figure 1: The Transformer - model architecture.

In this blog, we'll dive deep into the Transformer Decoder architecture specifically from the perspective of training. During training, the behavior of the decoder is slightly different compared to inference. While the architecture remains the same in both cases, during training, the decoder operates in a non-autoregressive manner, whereas in inference, it becomes autoregressive. This blog will focus entirely on the decoder's architecture during the training phase. The inference behavior will be covered in a subsequent post.

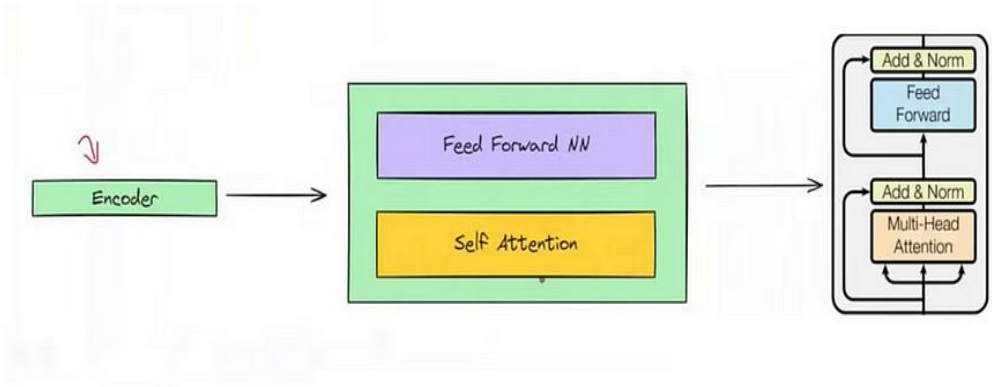
To simplify the understanding of the decoder, I'll begin by presenting an overview of the architecture and then gradually move into a detailed breakdown of each component and how they are interconnected. This step-by-step approach will ensure a clear understanding of the decoder's structure and functionality during training.

Now, imagine the Transformer as a large box containing two smaller boxes inside—an encoder and a decoder.



The encoder is responsible for processing the input data, while the decoder takes the encoder's output and generates the final output sequence. While this is a simplified version, the reality is that both the encoder and decoder consist of multiple layers, often six, stacked on top of each other. Each layer of the encoder and decoder follows the same architecture, but the parameters and weights within them vary.

The decoder, like the encoder, consists of multiple blocks, and understanding the structure of a single decoder block is crucial. **Each decoder block has two key components: a self-attention mechanism and a feed-forward neural network.**

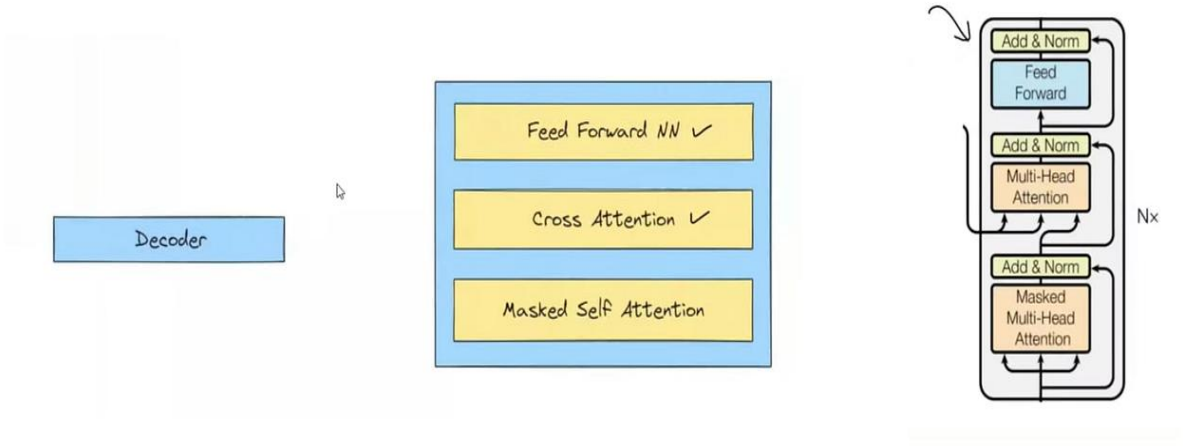


- **Internal Structure of single encoder block:**

While the architecture of these blocks is the same, their **internal parameters are different**, similar to how identical phones may have different apps installed for different users.

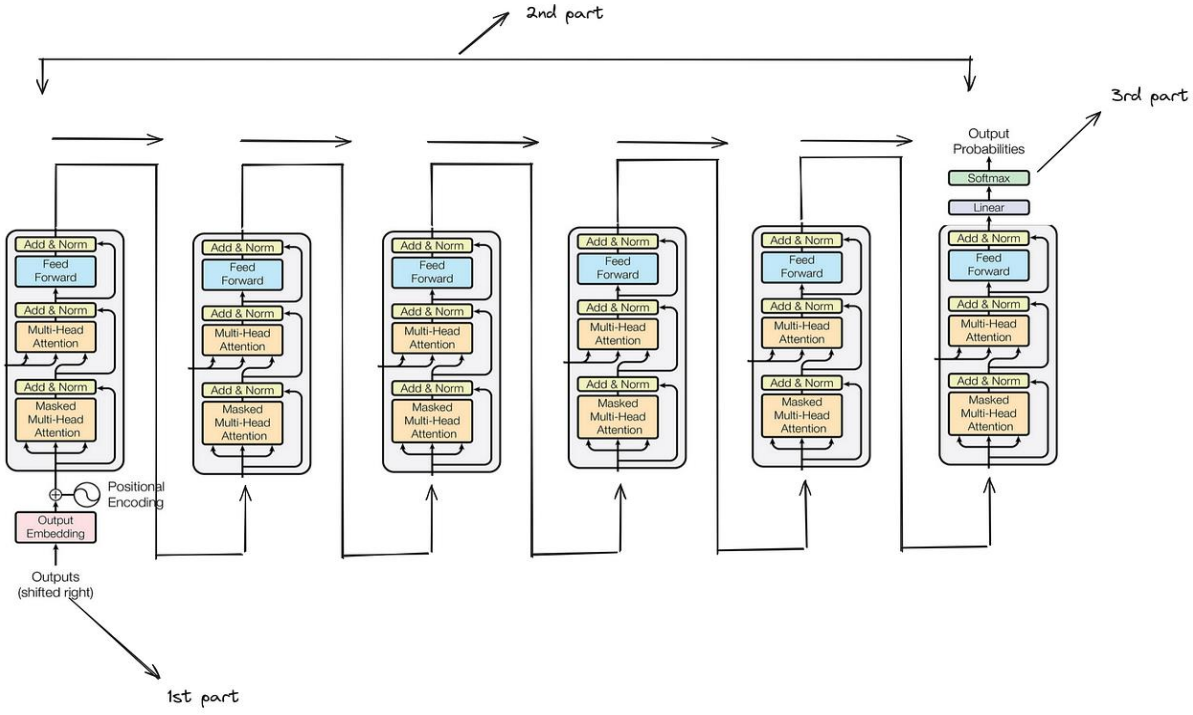
Let's now break down the decoder architecture. Much like the encoder, the decoder is composed of six stacked blocks. Each of these blocks consists of three key components: the

masked self-attention layer, the cross-attention layer (also known as the encoder-decoder attention), and the feed-forward neural network.



- **Internal Structure of single decoder block:**

Remember, we don't have just one decoder block; instead, there are six such blocks stacked sequentially. The output from the first decoder block becomes the input to the second, and so on, until we reach the sixth block. The final output from the sixth block is passed to the output layer, where we get the final prediction.



Flow of Information in 6 encoder block:

I know this might seem a bit overwhelming at first, but don't worry. We'll go through this architecture step by step, using an example that will make everything clearer. To simplify our exploration, I've broken it down into three parts:

- **Input preparation:** Before anything enters the decoder, it's essential to understand how the input is prepared. We'll discuss the steps involved in transforming the input so that it can be processed by the decoder.
- **Inside the decoder block:** This is where we'll spend the most time. We'll explore in detail what happens within a single decoder block. Once you understand the internal workings of one block, the same logic applies to all six, as their architectures are identical. The only differences are in their learned parameters.
- **Output generation:** Finally, we'll examine what happens in the output layer once the decoder has finished processing.

For our deep dive, we'll use a machine translation task as an example — specifically, translating from English to Hindi. Let's assume we're working with a training dataset containing English-Hindi sentence pairs. For simplicity, let's take a single example where the English sentence is "We are friends" and its corresponding Hindi translation is "हम दोस्त हैं".

Here's a critical point to remember: the encoder will have already processed the English sentence before the decoder even begins its work. The encoder's job is to generate contextual embeddings for each token in the sentence, which are then passed to the decoder. The decoder will use these embeddings as it generates the translated output, starting with the Hindi sentence.

1. Input Preparation :

Let's start with the input preparation phase of the decoder. This phase involves four key operations: shifting, tokenization, embedding, and positional encoding. The purpose of this input block is to take the output sentence (in our case, the Hindi sentence "हम दोस्त हैं") and process it so that it can be fed into the first block of the decoder.

Here's a more detailed explanation of these steps:

- **Right Shifting:** The first operation is right shifting. In this step, we add a special token called the start token at the beginning of the sentence. This start token acts as a flag, indicating the beginning of the training process. So, our transformed input now becomes: <START> हम दोस्त हैं.
- **Tokenization:** The next step is tokenization. This is where we break down the sentence into individual tokens. Tokenization can be done at various levels (words, bigrams, or n-grams), but for simplicity, we'll use word-level tokenization. After tokenizing, we get the following four tokens: <START>, हम, दोस्त, and हैं.
- **Embedding:** Once we have the tokens, the next step is to convert them into numerical representations that the machine can process. This is where the embedding layer comes in. The embedding layer takes each token and generates a

corresponding vector. In the original transformer paper, each vector has 512 dimensions.

So, for our tokens, we'll have the following embeddings:

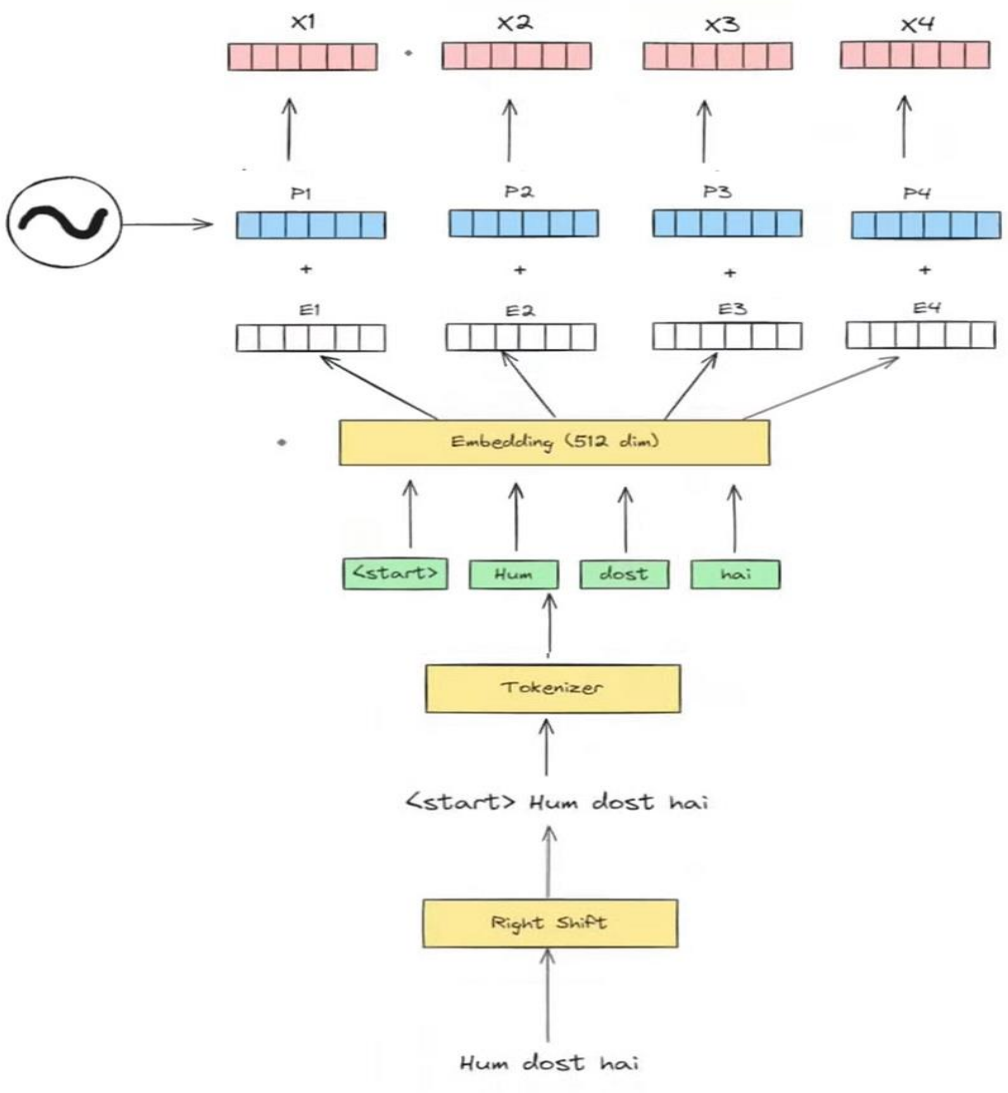
<START> corresponds to vector E1 (512-dimensional),

हम corresponds to vector E2 (512-dimensional),

दोस्त corresponds to vector E3 (512-dimensional),

हैं corresponds to vector E4 (512-dimensional).

At this point, we've successfully transformed our tokens into machine-readable vectors, but we still face one issue: we haven't encoded any information about the order of the tokens (i.e., which token comes first, second, etc.).



Input Preparation Workflow diagram for decoder block

- **Positional Encoding:** To address this, we use positional encoding, which helps the model understand the order of the words in the sentence. Positional encoding generates a unique vector for each position in the sentence. For example:
    Position 1 gets vector P1 (512-dimensional),
    Position 2 gets vector P2 (512-dimensional),
    Position 3 gets vector P3 (512-dimensional),
    Position 4 gets vector P4 (512-dimensional).

At this stage, we have **two sets of vectors**: the embedding vectors and the positional encoding vectors.
Next, we simply add these two sets of vectors together. For example:
The embedding vector for <START> (E1) is added to the positional vector for the first position (P1),
The embedding vector for हम (E2) is added to the positional vector for the second position (P2), and so on.
Since both sets of vectors have 512 dimensions, they can be added together easily. The result is our final set of input vectors: X1, X2, X3, and X4. These vectors correspond to the tokens in the sentence and encode both their meanings and their positions.
These final vectors, X1, X2, X3, and X4, are what we send into the first block of the decoder.
I hope this clarifies how the input block works and how the input sentence is prepared for processing in the decoder.

2. Inside the decoder block :

Now that we have our input vectors X1, X2, X3, and X4, we feed them into the decoder block. The first component to process these vectors is the masked multi-head attention block.
If you've read my blog on masked self-attention, you might recall that the masked multi-head attention works almost the same as regular multi-head attention. The only significant difference is the masking.
Let me quickly explain how this works. For each input vector (X1, X2, X3, X4), a corresponding contextual embedding vector is generated, like so:
Z1 is generated for X1,
Z2 is generated for X2,
Z3 is generated for X3,
Z4 is generated for X4.
However, the key difference here is that while generating Z1, we only consider the <START> token (X1) and ignore the rest (X2, X3, X4). While generating Z2, we consider both <START> and हम (X1 and X2) and ignore दोस्त and हैं (X3 and X4). Similarly, for Z3, we consider <START>, हम, and दोस्त (X1, X2, X3), but not हैं (X4). Finally, when generating Z4, we take all tokens into account (X1, X2, X3, X4).

This process is what makes it masked attention, as each vector is generated while only considering previous tokens, not future ones.

Now, if we look back at our main diagram, we can see that the output of the masked multi-head attention block is fed into the next layer: the add & normalize block.

In this block, the first operation is addition. The question is: what exactly are we adding? The answer is simple. We add the output of the masked multi-head attention block (Z1, Z2, Z3, Z4) to the original input vectors (X1, X2, X3, X4). This happens because of the residual connection or skip connection that bypasses the multi-head attention. The original input vectors are passed along this path and then added to the output of the multi-head attention. Looking at the diagram, you'll notice that X1, X2, X3, and X4 are passed not only into the multi-head attention but also through this residual connection. So now, we add:

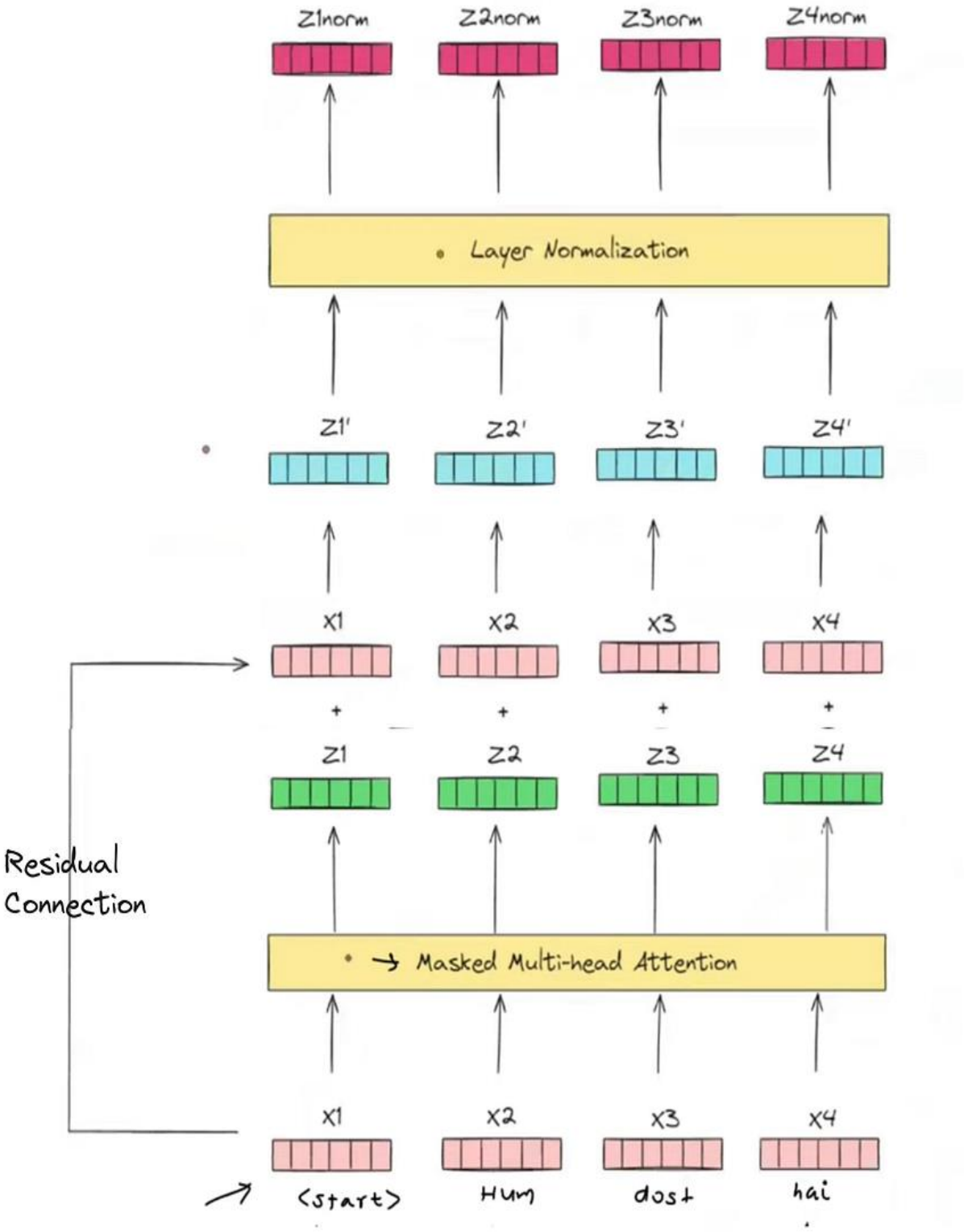
Z1 to X1,

Z2 to X2,

Z3 to X3,

Z4 to X4.

All of these vectors are 512-dimensional, so the addition operation is straightforward. After adding, we get a new set of vectors, which we can call Z1', Z2', Z3', and Z4'.



Next, we perform the normalization step. The type of normalization used here is layer normalization. What layer normalization does is calculate the **mean (μ)** and standard

deviation (σ) for each vector. Using these values, it normalizes each vector so that the values lie within a small, given range. This ensures that the training process remains stable. We use layer normalization because, during the previous operations (attention, additions, etc.), we might have generated large numbers, which could destabilize the training process. By normalizing the vectors, we ensure that the entire process remains stable.

At this point, we have reached the add & normalize block. The output from this step is ready, and we can now pass it to the next block in the decoder, which is the cross-attention block.

Now, let's move on to the cross-attention block, which is probably the most interesting part of the entire decoder architecture.

Here's why: the cross-attention block allows interaction between the input sequence (for example, an English sentence) and the output sequence (like a Hindi sentence). Essentially, for each token in your English sentence, you compute a similarity score with each token in your Hindi sentence. This is where the magic of cross-attention happens!

You'll notice that this block takes two inputs:

1. One input comes from the masked attention block, which we discussed earlier.
2. The second input comes from the encoder.

If you refer to the overall diagram, you'll see that once the encoder finishes its work, it passes its output to this stage in the decoder.

To summarize, the cross-attention block works just like a normal multi-head attention block, with one big difference: instead of a single input sequence, cross-attention uses two sequences.

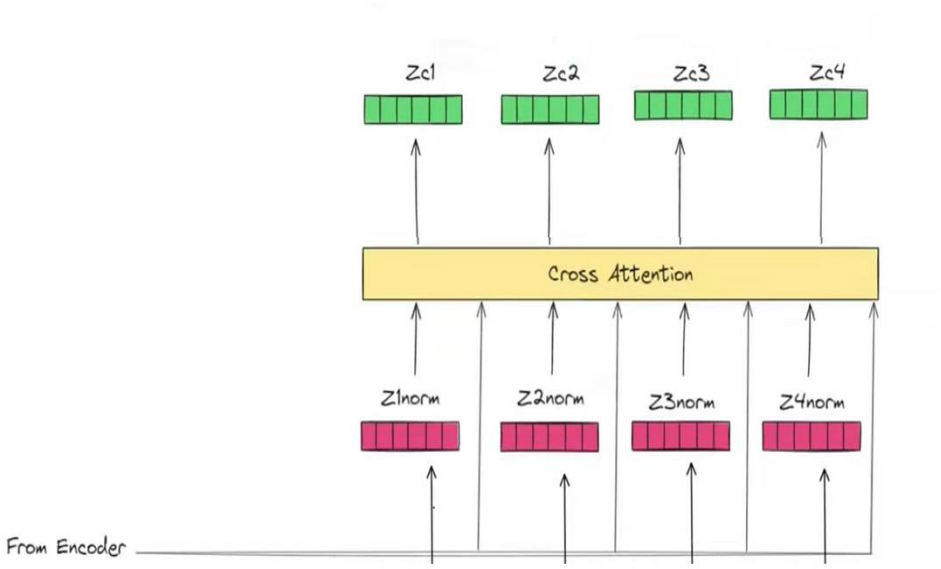
The first sequence is your English sentence (coming from the encoder).

The second sequence is your Hindi sentence (coming from the previous step of the decoder).

This is crucial because, for attention, you need three sets of vectors: **query, key, and value**.

The query vectors come from the decoder (based on the Hindi sentence).

The key and value vectors come from the encoder (based on the English sentence).



Once you have the query, key, and value vectors, the rest of the process is exactly like a normal self-attention block.

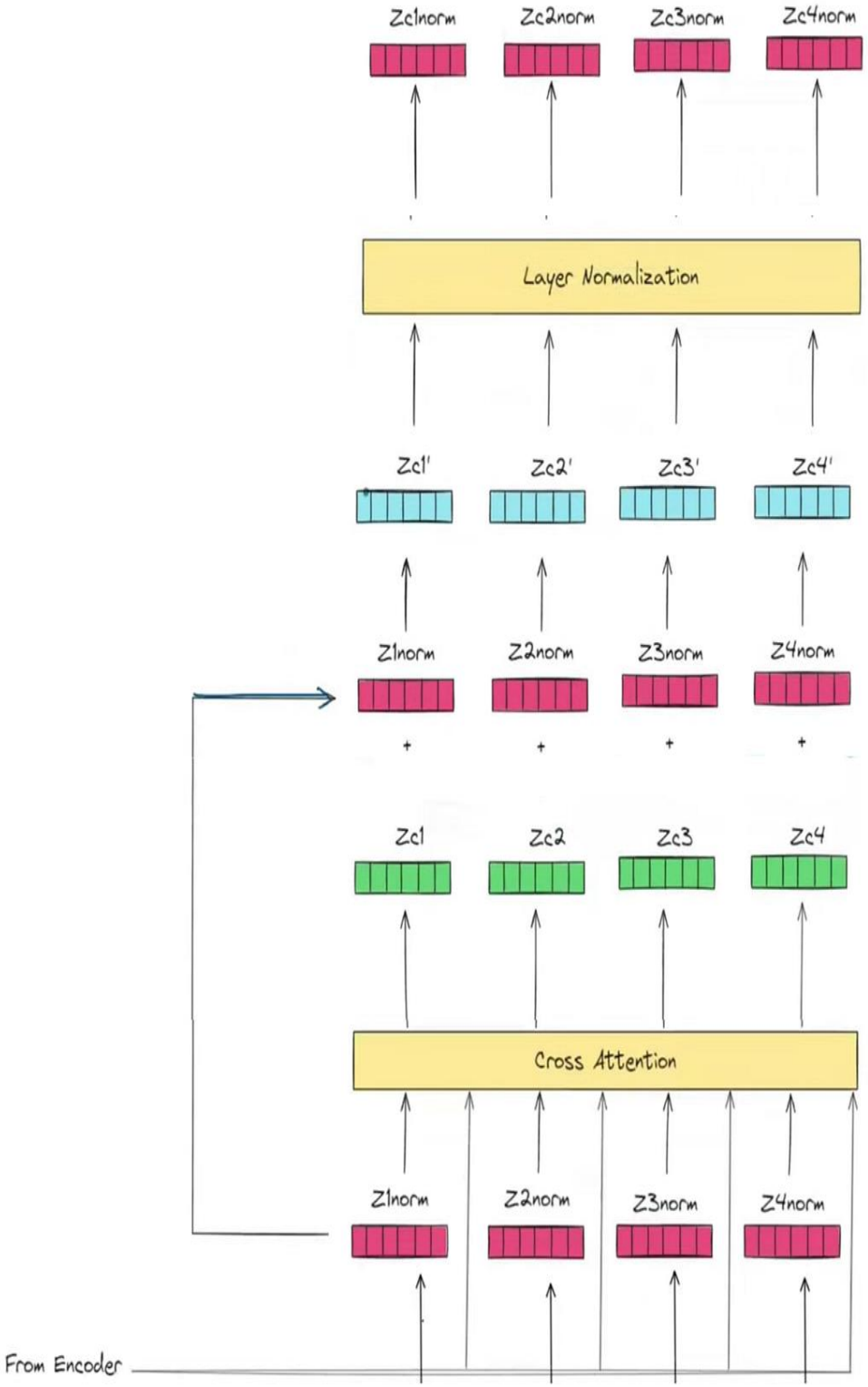
Let's look at the diagram again. Previously, we received Z1_norm, Z2_norm, Z3_norm, and Z4_norm from the masked attention block. Now, we'll send these vectors into the cross-attention block. But that's not all — we also feed the encoder embeddings into this block.

From the Z1_norm, Z2_norm vectors, we'll extract the query vectors. From the encoder embeddings, we'll extract the key and value vectors. Once you have all these vectors, the attention mechanism runs as usual. The result is a new contextual embedding vector for each token in your output sentence.

For example, if your output sentence has four tokens (as we do here), you'll get contextual embedding vectors for each token:

Zc1 for token 1,
Zc2 for token 2,
Zc3 for token 3,
Zc4 for token 4.

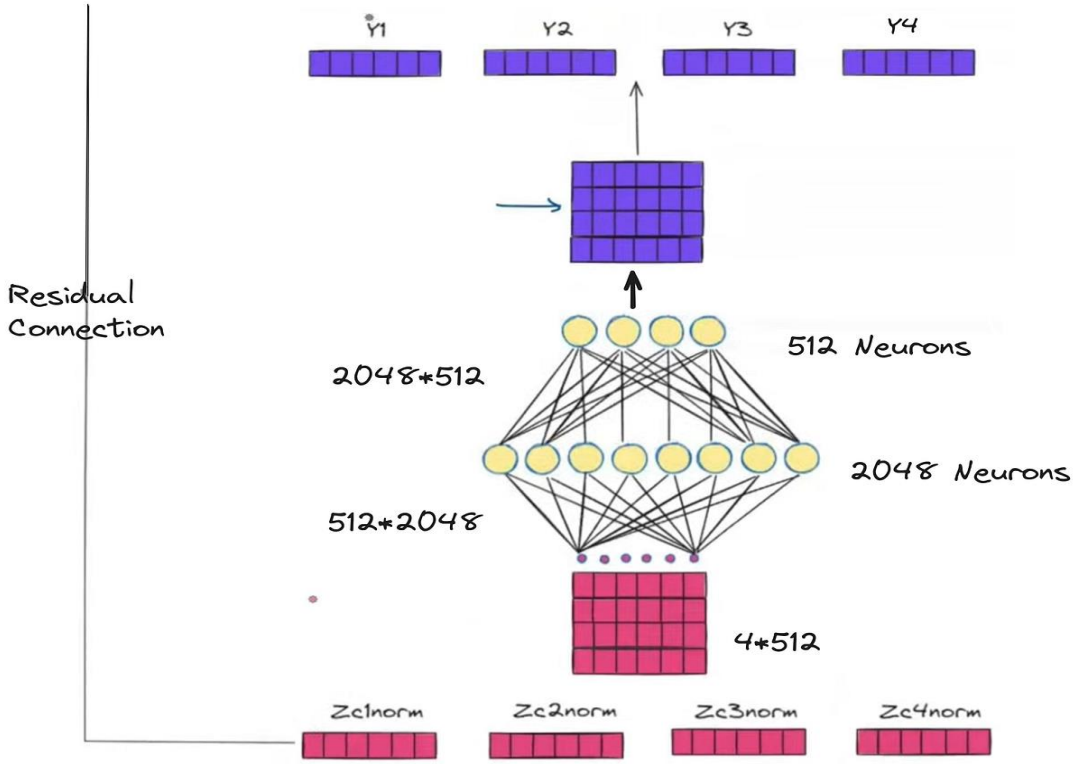
The "c" here stands for cross-attention, indicating that these embeddings are the output of the cross-attention block.



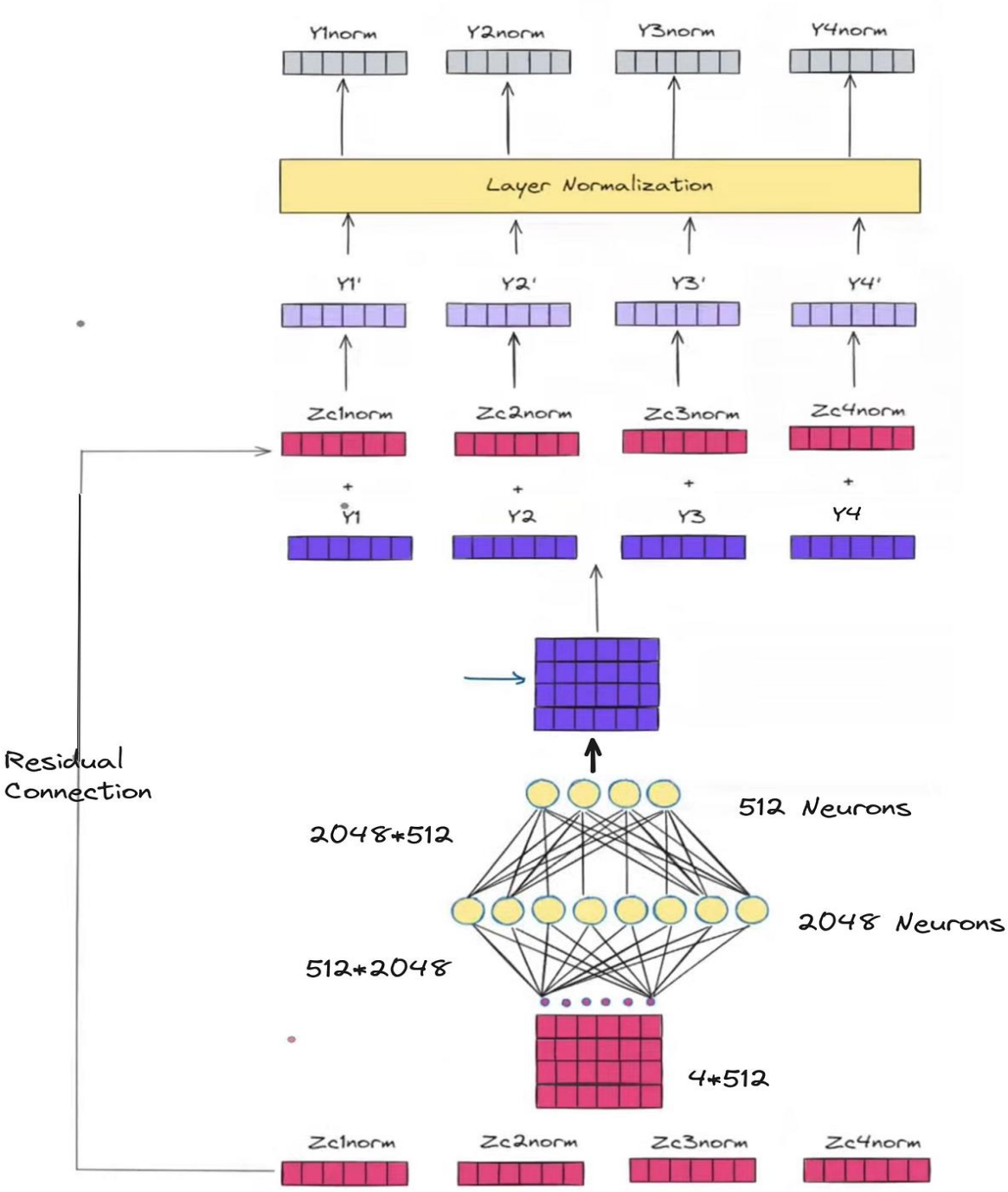
At this point, we have reached the cross-attention block's output, and once again, there is an add & normalize block right after it.

So, the question arises: what are we adding this time? It's simple — we add the cross-attention output (Zc1, Zc2, Zc3, Zc4) to the output of the previous masked attention step (Z1_norm, Z2_norm, etc.).
Notice that we not only sent Z1_norm, Z2_norm to the cross-attention block, but we also passed them through a residual connection to this point, where they are added to the cross-attention output. This is still fine because all vectors are 512-dimensional, so there's no problem with the addition operation.
After performing the addition, we get a new set of vectors, which we can call Zc1_norm, Zc2_norm, and so on.
The final step here is layer normalization, which ensures that the output vectors are normalized. After normalization, we get our new set of vectors Zc1_norm, Zc2_norm, and so on, which are ready to be passed to the next stage of the decoder.
At this point, we've reached the feed-forward block, which is the next component in our decoder journey.
At this point in our decoder architecture, we will now move on to the feed-forward layer. Now, these Zc1_norm, Zc2_norm, Zc3_norm and Zc4_norm vectors need to be passed through a feed-forward neural network.
Let me first explain the architecture of this feed-forward network. Interestingly, the architecture is exactly the same as the one we saw in the encoder.
This feed-forward neural network consists of two layers:
The first layer has 2048 neurons, and its activation function is ReLU.
The second layer has 512 neurons, and its activation function is linear.
When it comes to parameters, the feed-forward neural network expects an input of 512 dimensions. The first layer has weights of shape 512×2048 and biases of size 2048. We'll call these weights W1 and the biases b1 . Similarly, the second layer has weights of shape 2048×512 and biases of size 512, which we'll refer to as W2 and b2 .
Now, since we have four vectors of 512 dimensions each, we create a batch. By combining these vectors, we create a matrix of shape 4×512. This entire batch is passed through the feed-forward network simultaneously.
The first operation that takes place is the dot product of the input matrix Z with W1 , followed by adding the bias b1. Mathematically, this can be represented as:
$W1 \cdot Z + b1$
The resulting matrix has a shape of 4×2048. After this, we apply the ReLU activation function, which introduces non-linearity.
Next, the output is passed through the second layer, where we perform the dot product with W2 and add the bias b2. The result is a matrix of shape 4×512, giving us four vectors of size 512 each, just like we had before.

So, we started with four vectors of size 512, passed them through the feed-forward network, and ended up with four vectors of the same size. The key difference is that non-linearity has been introduced in the process due to the ReLU activation function.
After this operation, we now have four vectors, which we'll call y1,y2,y3,y4 .



At this point, we are here in the diagram, at the output of the feed-forward block. Again, there's an add and normalize operation ahead. So, we take the output of the feed-forward network and add it to the original input zc1norm, zc2norm , and so on, using a residual connection. Since all these vectors are of size 512, we can perform element-wise addition. After performing the addition, we normalize the result using layer normalization, as we've done after every major operation. This gives us the final set of vectors y1norm, y2norm, y3norm, y4norm , which are still 512-dimensional vectors.



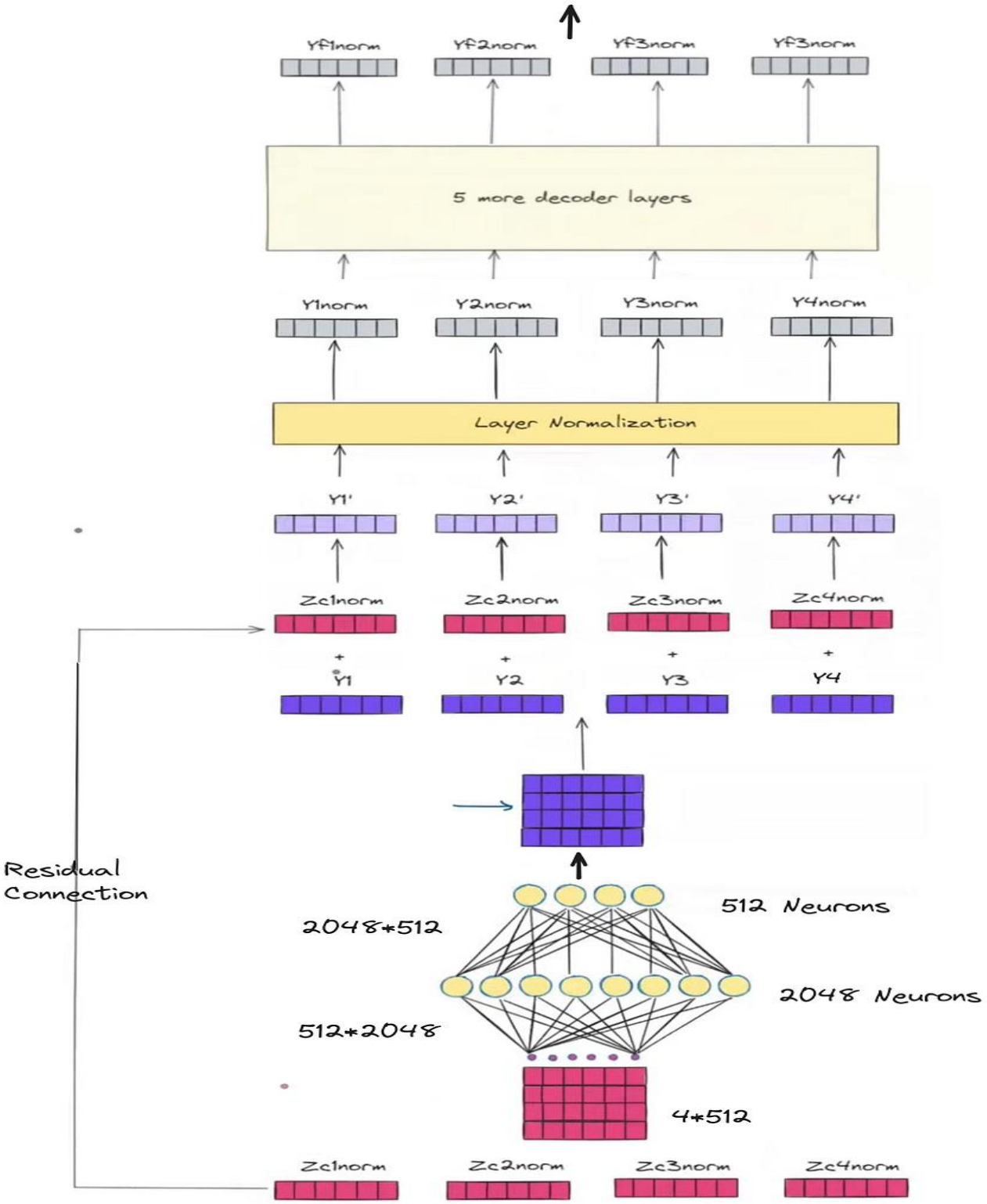
With this, we have reached this point in the decoder, and these four vectors are the output of the first decoder block.

I hope this explanation makes it clear how a single decoder block functions from start to finish!

As you already know, we don't just have one decoder block. In total, there are six decoder blocks. So, once you get the output from the first decoder block, which is y1norm, y2norm, y3norm, and so on, you send this output directly to the second decoder block. In the second

decoder block, the same operations will be executed as in the first block. The only difference is that the parameters will be different, but the operations will remain identical. Again, you will apply masked multi-head attention, followed by normalization. After that, you'll apply cross attention and normalize again. Then, you'll pass the result through the feed-forward layer, followed by another normalization step. After these operations, you'll reach this point, and you'll get another set of vectors, similar to how you got y1norm, y2norm and so on from the first block. These vectors will then be passed on to the third decoder block.

This process continues until you reach the sixth decoder block. Finally, the output of the sixth decoder block will be the final output.

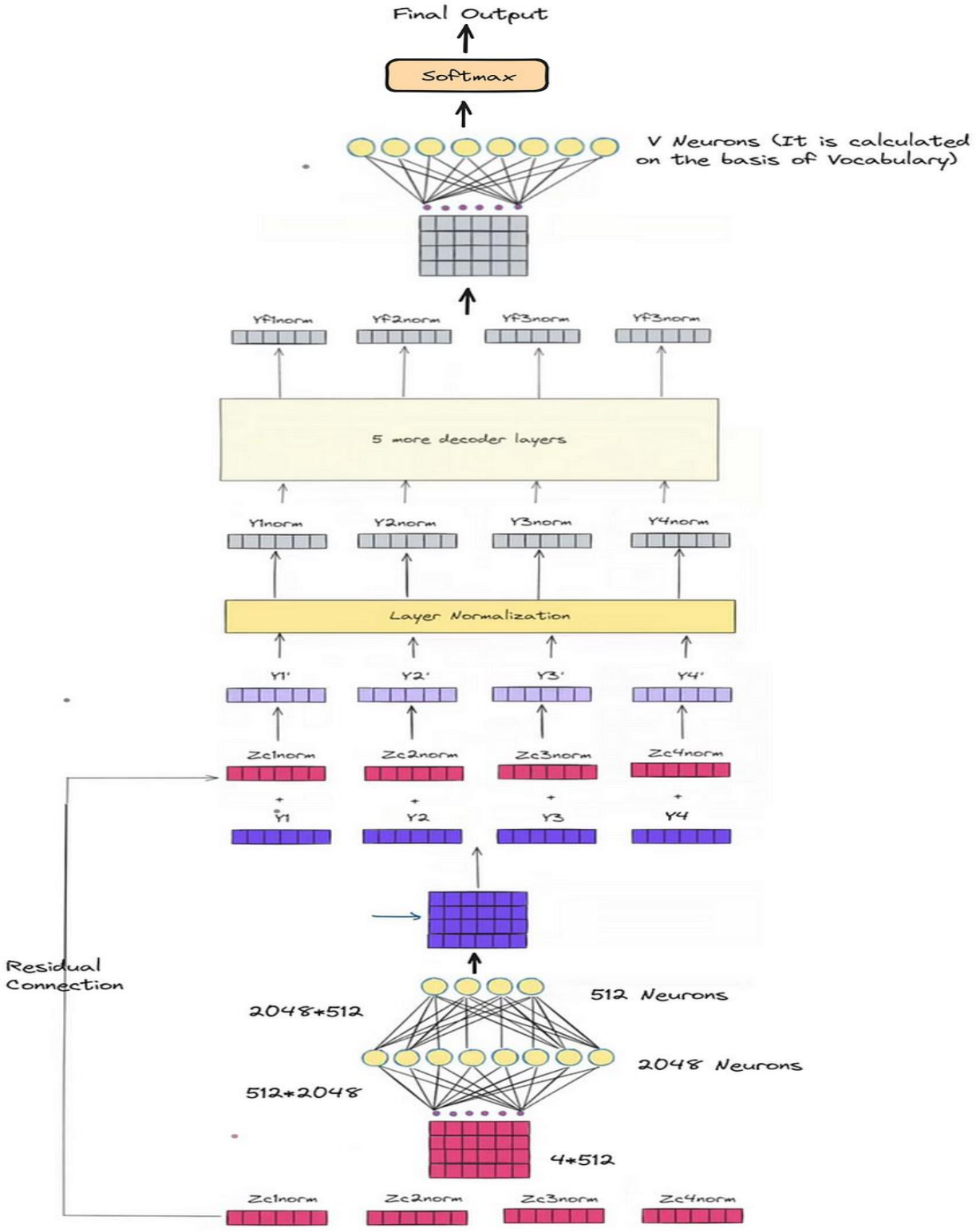


In the diagram, I've shown that you're currently in decoder one, and you've just received the output of the first decoder block. After that, there are five more decoder layers, or blocks, to process. As you go through each of these blocks, you will eventually get the final

output, which I've denoted as yf1norm, yf2norm, yf3norm and so on, where "f" stands for "final."

3. Output generation :

To understand how this works, we need to look at the output portion, which consists of two parts: a **Linear Layer** and a **Softmax Layer.** This is similar to the output layer of a feed-forward neural network.



Here's how it works:

You have a single layer that takes a 512-dimensional input. The number of neurons in this layer is V, where V is a predefined number based on the vocabulary size of the Hindi words in your dataset. Let's break this down:

We're translating from English to Hindi. On one side, you have English sentences, and on the other side, you have their corresponding Hindi translations. For example, let's take a dataset of 5000 sentence pairs. You'll go through the Hindi side of the dataset and count all the unique words. For example, "बढ़िया" is a unique word, "हम" is a unique word, "दोस्त" is a unique word, and so on. You won't count a word again if it has already appeared, like "हम" being repeated in another sentence.

Once you have the unique words, you form the vocabulary for the Hindi words. Suppose, in your dataset of 5000 Hindi sentences, there are 10,000 unique words. This 10,000 will be the value of V. The number of neurons in this output layer will be equal to the size of your Hindi vocabulary, i.e., 10,000 neurons.

Each neuron represents one unique Hindi word. The first neuron corresponds to the first word in your vocabulary, such as "मैं," the second neuron corresponds to "बढ़िया," and so on, until the 10,000th neuron.

Now, how are the weights structured?

Each input has 512 dimensions.

There are 10,000 neurons in the output layer.

Thus, you'll have weights of size 512 x 10,000 and also 10,000 bias values (one for each neuron).

At this point, you have four vectors — one for each token. For instance:

First vector for the token "स्टार्ट"

Second vector for the token "हम"

Third vector for the token "दोस्त"

Fourth vector for the token "है"

Since the input to this layer can only be in 512 dimensions, you stack these four vectors into a matrix of size 4 x 512. You can now pass this entire matrix into the layer in batch form, enabling parallel processing for all tokens. But for simplicity, let's focus on just one token.

Assume we send only the vector corresponding to the "स्टार्ट" token into this layer. The input would be of size 1 x 512. What happens next?

The vector yf1normy is multiplied by the weights, W3. Here, W3 is of size 512 x 10,000. The result will be a vector of size 1 x 10,000, giving you 10,000 unique numbers — one for each neuron. Each neuron will output a number. For example:

Neuron 1 outputs 3

Neuron 2 outputs 6

Neuron 3 outputs some other value, and so on.

These values are not normalized yet, so the next step is to apply Softmax, which normalizes the numbers such that their sum equals 1. This forms a probability distribution. For example, after applying Softmax, you might get:
Neuron 1 (word: “मैं”) has a probability of 0.01

Neuron 2 (word: “बढ़िया”) has a probability of 0.02

Neuron 3 (word: “हम”) has the highest probability, say 0.25.

Since “हम” has the highest probability, it will be chosen as the output word for the “स्टार्ट” token.

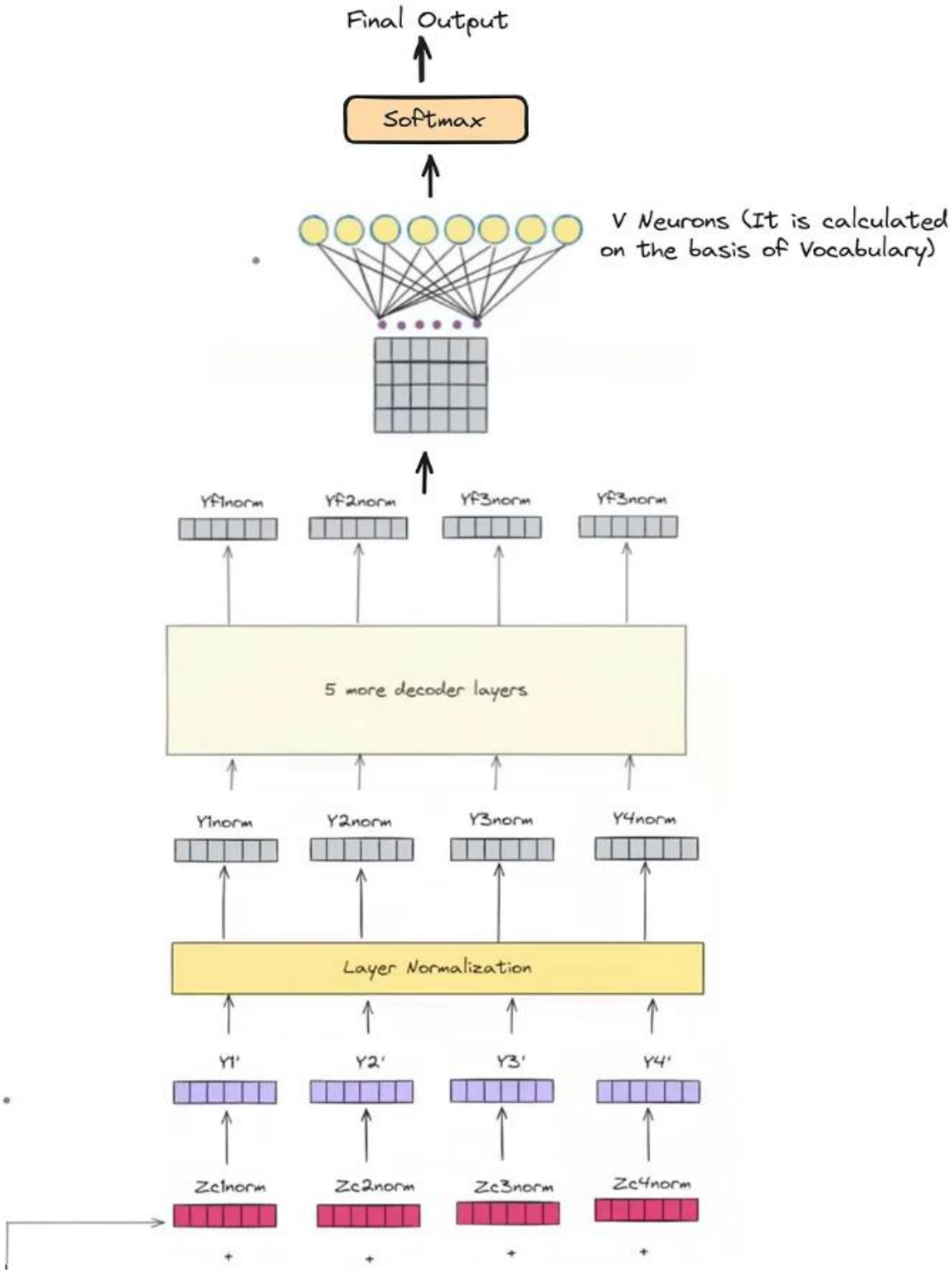
Next, you do the same for the other vectors yf2norm,yf3norm, and so on, multiplying them by the weights W3 and adding the bias, then applying Softmax to get the output for each token.

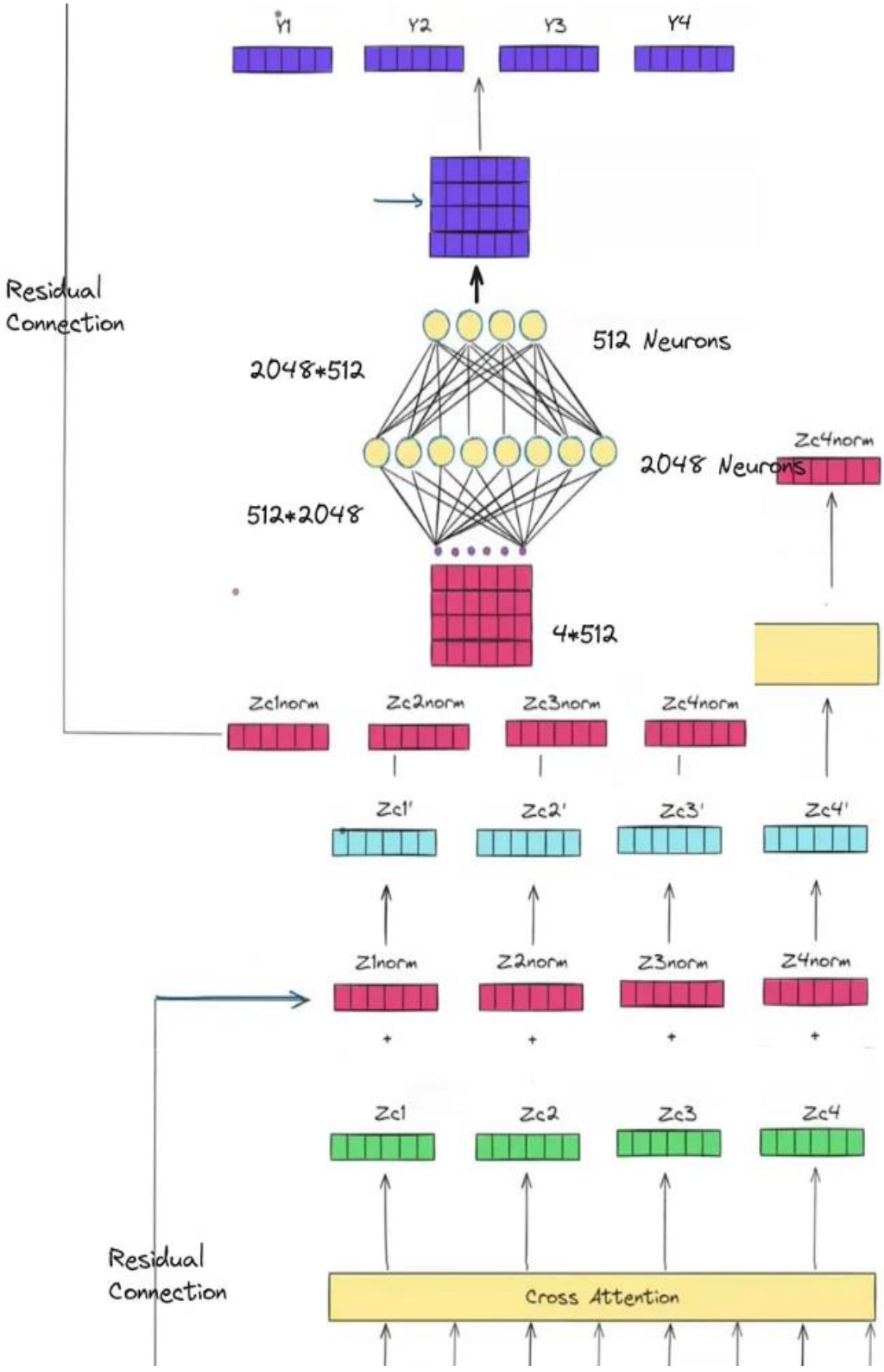
For example:

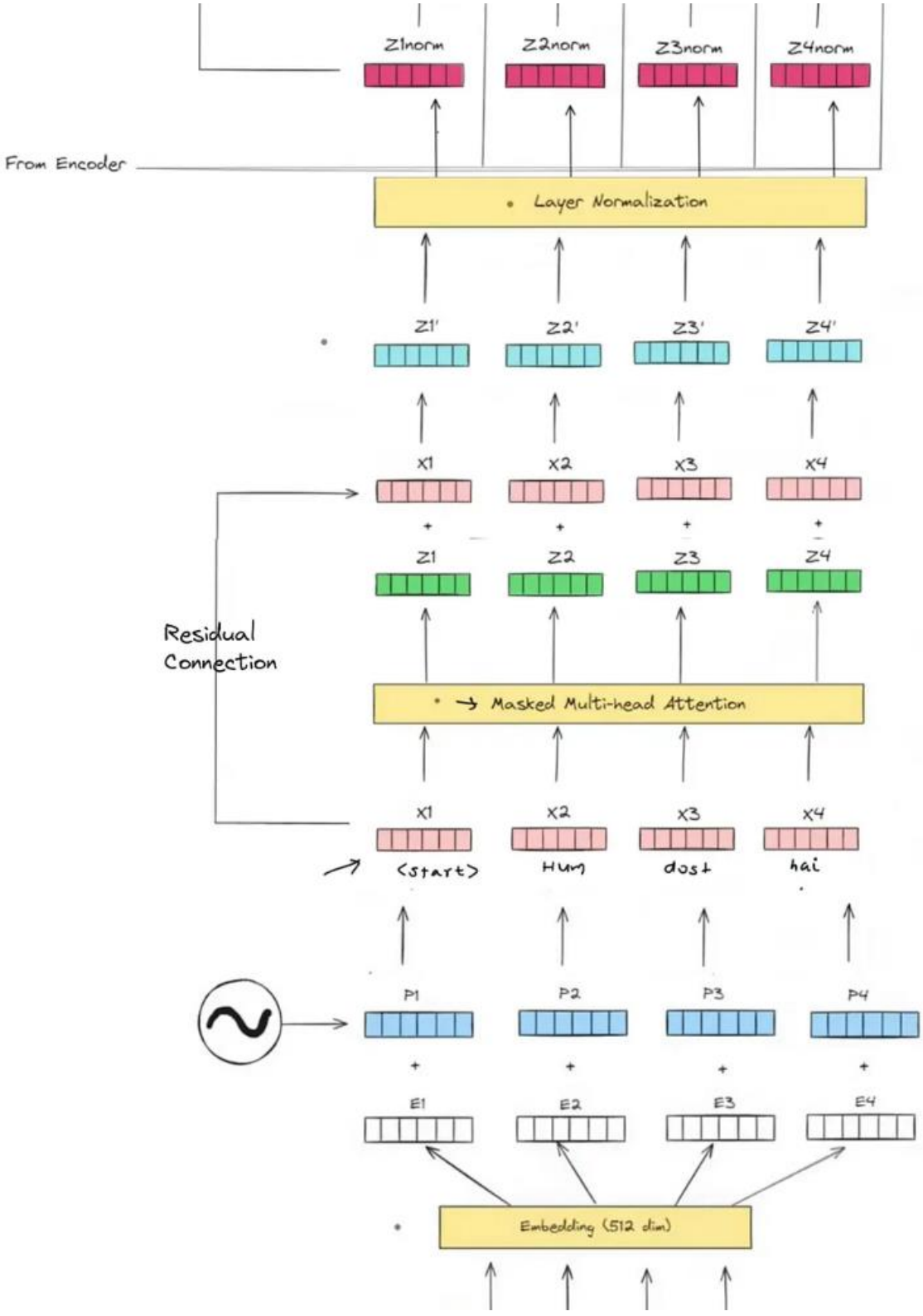
yf2norm might correspond to the word “दोस्त” with the highest probability.

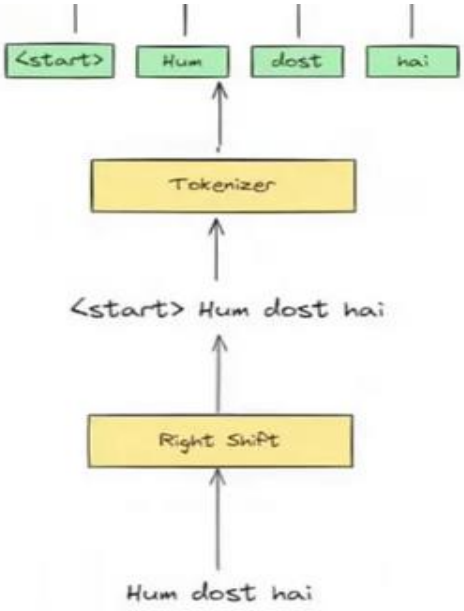
Thus, the final decoded sentence could be something like: “हम दोस्त हैं”.

This is how the decoder architecture works. I hope this explanation makes things clear!


Final Output
Softmax
V Neurons (It is calculated on the basis of Vocabulary)
Yf1norm
Yf2norm
Yf3norm
Yf3norm
5 more decoder layers
Y1norm
Y2norm
Y3norm
Y4norm
Layer Normalization
Y1'
Y2'
Y3'
Y4'
Zc1norm
Zc2norm
Zc3norm
Zc4norm
+
+
+
+


Y1
Y2
Y3
Y4
Residual Connection
512 Neurons
2048*512
2048 Neurons
Zc4norm
512*2048
4*512
Zc1norm
Zc2norm
Zc3norm
Zc4norm
Zc1'
Zc2'
Zc3'
Zc4'
Z1norm
Z2norm
Z3norm
Z4norm
+
+
+
+
Zc1
Zc2
Zc3
Zc4
Residual Connection
Cross Attention


Z1norm
Z2norm
Z3norm
Z4norm
From Encoder
Layer Normalization
Z1'
Z2'
Z3'
Z4'
X1
X2
X3
X4
Z1
Z2
Z3
Z4
Residual Connection
Masked Multi-head Attention
X1
X2
X3
X4
<start>
Hum
dost
hai
P1
P2
P3
P4
E1
E2
E3
E4
Embedding (512 dim)


<start>
Hum
dost
hai
Tokenizer
<start> Hum dost hai
Right Shift
Hum dost hai